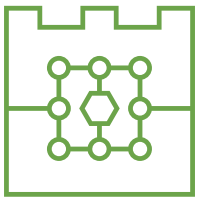


Politechnika Krakowska
im. Tadeusza Kościuszki



Wydział Informatyki
i Matematyki

Politechnika krakowska im. Tadeusza Kościuszki w Krakowie

Wydział Informatyki i Matematyki

Sprawozdanie z projektu ZTKIK

Projekt 5: Automatyczne łamanie szyfrów podstawieniowych za pomocą algorytmu genetycznego

Autorzy: Radosław Rolka (CY3), Weronika Wojtas (CY4)
Studia: Informatyka-Cyberbezpieczeństwo, II stopień, II semestr

Kraków, 2026

Spis treści

Streszczenie **4**

1 Wprowadzenie **5**
1.1 Kontekst . . . 5
1.2 Cel i zakres . . . 5
1.3 Struktura sprawozdania . . . 5

2 Formalny model zagrożeń **6**
2.1 Badany system . . . 6
2.2 Profil przeciwnika . . . 6
2.3 Interfejs dostępny dla przeciwnika . . . 6
2.4 Zasada Kerckhoffsa . . . 6
2.5 Ilościowe kryterium sukcesu . . . 7
2.6 Założenia i zakres . . . 7

3 Metodyka **8**
3.1 Dane i ich podział . . . 8
3.2 Generowanie szyfrogramów . . . 9
3.3 Model n-gramowy i funkcja przystosowania . . . 9
3.4 Algorytm genetyczny . . . 9
3.5 Metryki i protokół . . . 10

4 Implementacja **11**
4.1 Struktura kodu . . . 11
4.2 Wydajność funkcji przystosowania . . . 11
4.3 Fragmenty kodu . . . 11
4.4 Odtwarzalność . . . 13

5 Wyniki **14**
5.1 Próba ataku według kryterium projektu . . . 14
5.2 Wpływ długości szyfrogramu . . . 15
5.3 Rząd n-gramów w funkcji przystosowania . . . 16
5.4 Szyfrogram z zachowanymi odstępami . . . 17
5.5 Przykład działania: szyfrowanie i atak . . . 17

6 Wnioski **19**
6.1 Ograniczenia . . . 19

Bibliografia **20**

A Macierz wkładu **21**

Spis rysunków **22**

Spis tabel **23**

Streszczenie

Praca przedstawia implementację i ocenę eksperymentalną ataku ze znanym szyfrogramem (COA) na monoalfabetyczny szyfr podstawieniowy z użyciem algorytmu genetycznego (GA) zrealizowanego we frameworku DEAP. Osobnikiem jest permutacja 26 liter (klucz deszyfrujący), a funkcją przystosowania – logarytmiczna wiarygodność odszyfrowanego tekstu w modelu 4-gramowym języka, wyestymowanym z książek z domeny publicznej (Project Gutenberg dla języka angielskiego, Wolne Lektury dla polskiego). Zgodnie z zasadą Kerckhoffsa jawne są algorytm, kod i model językowy. Tajny jest wyłącznie klucz (przestrzeń $26! \approx 2^{88}$). Ocenę przeprowadzono na 50 szyfrogramach angielskich i 50 polskich o długości 500 znaków, pochodzących z książek rozłącznych z korpusem treningowym. Kryterium sukcesu projektu – poprawność literowa odczytu powyżej 95 % – spełniono w 96 % prób (EN) i 80 % (PL), przy średniej poprawności 0,966 i 0,855 oraz czasie około sekundy na szyfrogram. Zbadano także wpływ długości szyfrogramu, rzędu n-gramów w funkcji przystosowania oraz jawnych odstępów między słowami.

Słowa kluczowe: kryptoanaliza, szyfr podstawieniowy, algorytm genetyczny, DEAP, model n-gramowy, atak ze znanym szyfrogramem.

Rozdział 1

Wprowadzenie

1.1. Kontekst

Monoalfabetyczny szyfr podstawieniowy zastępuje każdą literę tekstu jawnego inną literą według tajnej permutacji alfabetu. Mimo ogromnej przestrzeni kluczy szyfr jest słaby, ponieważ zachowuje statystykę języka: każdej literze jawnej odpowiada zawsze ta sama litera szyfrogramu, więc rozkłady liter i n-gramów są jedynie spermutowane. Ręczna kryptoanaliza (analiza częstości, odgadywanie słów) jest żmudna, dlatego automatyzuje się ją metaheurystykami, w tym algorytmami ewolucyjnymi [1].

1.2. Cel i zakres

Celem projektu jest implementacja w języku Python (framework DEAP [2]) algorytmu genetycznego, który na podstawie samego szyfrogramu odtwarza klucz szyfru podstawieniowego, oraz eksperymentalna ocena ataku. Etapy realizacji:

1. wygenerowanie szyfrogramów podstawieniowych z tekstów z wolnych bibliotek
2. funkcja przystosowania oparta na częstościach n-gramów
3. konfiguracja krzyżowania i mutacji w DEAP
4. ewolucyjne odtworzenie tekstu i pomiar metryk

1.3. Struktura sprawozdania

- rozdział 2 definiuje formalny model zagrożeń
- rozdział 3 opisuje dane, generowanie szyfrogramów, funkcję przystosowania, algorytm genetyczny i metryki
- rozdział 4 opisuje implementację
- rozdział 5 przedstawia wyniki próby ataku i eksperymentów dodatkowych
- rozdział 6 zawiera wnioski i ograniczenia

Rozdział 2

Formalny model zagrożeń

2.1. Badany system

Alfabet $\mathscr{A} = \{\mathtt{A}, \ldots, \mathtt{Z}\}$ utożsamiamy z $\mathbb{Z}_{26}$. Kluczem jest permutacja $\pi\colon \mathbb{Z}_{26} \to \mathbb{Z}_{26}$, a szyfrowanie i deszyfrowanie to $e_\pi(x) = \pi(x)$ oraz $d_\pi(y) = \pi^{-1}(y)$, stosowane litera po literze. Tekst jawny jest tekstem w znanym języku naturalnym (angielskim lub polskim), sprowadzonym do 26 wielkich liter (polskie znaki diakrytyczne zastąpiono literami podstawowymi, interpunkcję i cyfry usunięto). Rozważamy dwa warianty: szyfrogram złożony z samych liter (wariant podstawowy, trudniejszy) oraz szyfrogram z jawnymi odstępami między słowami.

2.2. Profil przeciwnika

Przeciwnik jest **bierny**: obserwuje przechwycony szyfrogram, ale nie może wpływać na nadawcę, odbiorcę ani na przesyłaną wiadomość. Dysponuje zwykłym komputerem klasy PC i czasem rzędu od sekund do minut na jeden szyfrogram.

2.3. Interfejs dostępny dla przeciwnika

Model dostępu to **atak ze znanym szyfrogramem (COA)**: przeciwnik zna wyłącznie pojedynczy szyfrogram $y = e_\pi(x)$ o długości L znaków oraz język tekstu jawnego.

2.4. Zasada Kerckhoffsa

Zakładamy pełną jawność: algorytmu szyfru, kodu źródłowego ataku (pakiet gasub), użytych korpusów i wyestymowanego modelu n-gramowego oraz wszystkich hiperparametrów GA. Bezpieczeństwo systemu zależy wyłącznie od tajności klucza π, którego przestrzeń liczy $26! \approx 4{,}03 \cdot 10^{26} \approx 2^{88{,}4}$ elementów – przegląd zupełny jest więc niewykonalny i atak musi wykorzystać redundancję języka.

2.5. Ilościowe kryterium sukcesu

Dla pojedynczej próby (jeden szyfrogram, jeden przebieg ataku) definiujemy *poprawność literową*

$$\mathrm{Acc} = \frac{1}{L'} \sum_{i=1}^{L'} [d_{\hat{\pi}}(y_i) = x_i], \tag{2.1}$$

gdzie L' jest liczbą pozycji literowych (odstępy pomijamy), $\hat{\pi}$ – kluczem znalezionym przez atak, a $[\cdot]$ – indykatorem. Próba jest **sukcesem**, gdy $\mathrm{Acc} > 0{,}95$ dla szyfrogramu o długości $L = 500$ znaków (kryterium z opisu projektu). Na poziomie eksperymentu raportujemy średnią, odchylenie standardowe, medianę i minimum Acc oraz **odsetek sukcesów** w N niezależnych próbach. Atak uznajemy za skuteczny, gdy odsetek sukcesów wynosi co najmniej 90 %. Metryka pomocnicza to wskaźnik $F \geq F^*$ – odsetek prób, w których znaleziony klucz ma przystosowanie nie gorsze niż klucz prawdziwy (pozwala odróżnić porażki *przeszukiwania* od porażek *modelu językowego*).

2.6. Założenia i zakres

Poza zakresem pozostają szyfry homofoniczne i polialfabetyczne, teksty w języku innym niż model, teksty bardzo krótkie (poniżej 50 znaków) oraz szyfrogramy z zachowaną interpunkcją i wielkością liter.

Rozdział 3

Metodyka

3.1. Dane i ich podział

Teksty pochodzą z dwóch wolnych bibliotek: Project Gutenberg [3] (język angielski) i Wolne Lektury [4] (język polski). Dla każdego języka wybrano rozłączne zbiory książek: *treningowy*, z którego estymowany jest model n-gramowy, i *testowy*, z którego losowane są teksty jawne do zaszyfrowania (tabela 3.1). Z pobranych plików usuwane są nagłówki i stopki wydawcy, a tekst jest normalizowany: dekompozycja Unicode i usunięcie znaków diakrytycznych, zamiana na wielkie litery, usunięcie wszystkich znaków spoza A–Z (w wariancie z odstępami ciągi znaków niebędących literami zastępuje pojedynczy odstęp).

Tabela 3.1: Korpusy tekstowe użyte w eksperymentach (liczba liter po normalizacji do 26 liter).

język	zbiór	książek	liter	tytuły
angielski (Project Gutenberg)	treningowy	6	5 881 643	Austen, Pride and Prejudice; Melville, Moby Dick; Dickens, Great Expectations; Tolstoy, War and Peace; Dickens, A Tale of Two Cities; Bronte, Wuthering Heights
angielski (Project Gutenberg)	testowy	6	2 122 762	Carroll, Alice's Adventures in Wonderland; Doyle, The Adventures of Sherlock Holmes; Shelley, Frankenstein; Stoker, Dracula; Stevenson, Treasure Island; Wilde, The Picture of Dorian Gray
polski (Wolne Lektury)	treningowy	7	4 090 420	Mickiewicz, Pan Tadeusz; Prus, Lalka, t. 1; Prus, Lalka, t. 2; Sienkiewicz, Quo vadis; Sienkiewicz, Potop, t. 1; Sienkiewicz, Ogniem i mieczem, t. 1; Reymont, Chłopi, cz. 1
polski (Wolne Lektury)	testowy	6	2 608 752	Sienkiewicz, W pustyni i w puszczy; Sienkiewicz, Krzyżacy, t. 1; Prus, Faraon, t. 1; Żeromski, Przedwiośnie; Żeromski, Syzyfowe prace; Reymont, Ziemia obiecana, t. 1

3.2. Generowanie szyfrogramów

Jedno *zadanie* to para (tekst jawny, szyfrogram): z losowo wybranej książki testowej losowany jest fragment o zadanej długości L, a następnie losowana jest permutacja π i fragment jest szyfrowany. Ocena końcowa używa ziaren 0–49, a dobór parametrów GA wykonano na ziarnach od 1000 wzwyż, czyli na innych fragmentach i innych kluczach.

3.3. Model n-gramowy i funkcja przystosowania

Niech $c(g)$ będzie licznością n-gramu g w korpusie treningowym, N – łączną liczbą n-gramów, a $V = 26$ (lub 27 z odstępem) rozmiarem alfabetu. Prawdopodobieństwo wygładzone addytywnie i jego logarytm wynoszą

$$P(g) = \frac{c(g) + \alpha}{N + \alpha V^n}, \qquad \alpha = 0{,}01. \tag{3.1}$$

Dla klucza deszyfrującego d i szyfrogramu y funkcja przystosowania to log-wiarygodność odszyfrowanego tekstu $t = d(y)$:

$$F(d) = \sum_{i=1}^{|t|-n+1} \log_{10} P\big(t_i t_{i+1} \ldots t_{i+n-1}\big), \tag{3.2}$$

maksymalizowana przez GA. Wartość $F^* = F(\pi^{-1})$ dla klucza prawdziwego jest punktem odniesienia: jeśli atak znajdzie klucz $\hat{d}$ z $F(\hat{d}) \geq F^*$, ale Acc < 1, to porażka obciąża model językowy, a $F(\hat{d}) < F^*$ oznacza przedwczesną zbieżność. Model jest przechowywany jako tablica V^n liczb zmiennoprzecinkowych. Dla $n = 4$ i $V = 26$ to 456 976 wartości.

3.4. Algorytm genetyczny

Osobnik to lista 26 indeksów – permutacja, w której gen na pozycji y określa literę jawną przypisaną literze szyfrogramu y (klucz deszyfrujący). Taka reprezentacja czyni każdą permutację osobnikiem dopuszczalnym.

- **inicjalizacja** – losowe permutacje
- **selekcja turniejowa** – z k losowo wybranych osobników do rozrodu przechodzi najlepszy
- **krzyżowanie PMX** – segment między dwoma losowymi punktami jest wymieniany między rodzicami, a konflikty są naprawiane przez odwzorowanie wymienionych genów
- **mutacja** – każdy gen z prawdopodobieństwem $p_{\text{gen}} = 2/26$ jest zamieniany z losowym innym genem
- **elityzm** – dwa najlepsze osobniki przechodzą do następnego pokolenia bez zmian

Pętla ewolucyjna jest zaimplementowana jawnie, aby rejestrować w każdym pokoleniu najlepsze i średnie przystosowanie oraz najlepszy klucz (krzywe zbieżności). Wartości z tabeli 3.2 dobrano we wstępnych przebiegach na osobnym zbiorze ziaren, przy stałym budżecie około 60 000 ocen funkcji przystosowania na przebieg.

Tabela 3.2: Konfiguracja algorytmu genetycznego.

element	wartość	implementacja (DEAP)
reprezentacja	permutacja 26 indeksów (klucz deszyfrujący)	creator.Individual(list)
populacja	400 osobników	random.sample
liczba pokoleń	150 (budżet $\approx 60\,000$ ocen)	pętla własna + algorithms.varAnd
selekcja	turniejowa, rozmiar turnieju 5	tools.selTournament
krzyżowanie	PMX, $p_c = 0,8$	tools.cxPartialyMatched
mutacja	zamiana genów, $p_m = 0,6$, $p_{\text{gen}} = 2/26$	tools.mutShuffleIndexes
elityzm	2 najlepszych osobników	tools.selBest + HallOfFame
inicjalizacja	losowe permutacje	–
funkcja przystosowania	log-wiarygodność 4-gramów	FitnessFunction (NumPy)

3.5. Metryki i protokół

Dla każdej próby rejestrowane są: poprawność literowa, poprawność klucza, sukces (Acc $>$ 0,95), $F(\hat{d})$ i F^*, liczba ocen funkcji przystosowania, liczba pokoleń, czas oraz krzywa zbieżności (poprawność najlepszego osobnika po każdym pokoleniu). Próba ataku według kryterium projektu obejmuje $N = 50$ niezależnych prób na język (ziarna 0–49: różne fragmenty, klucze i przebiegi losowe) dla szyfrogramów o długości 500 znaków złożonych z samych liter. Raportujemy średnią z odchyleniem standardowym, medianę, minimum i odsetek sukcesów. Dodatkowo, przy tej samej konfiguracji GA i $N = 30$ próbach na język (ziarna 0–29), zbadano wpływ długości szyfrogramu (50–1000 znaków), rzędu n-gramów w funkcji przystosowania ($n = 1,\ldots,5$) oraz jawnych odstępów między słowami (model z 27-symbolowym alfabetem).

Rozdział 4

Implementacja

4.1. Struktura kodu

Projekt (Python 3.10 lub nowszy, biblioteki deap 1.4, numpy 1.24 i matplotlib 3.7 lub nowsze) ma budowę modułową:

- gasub/data.py – pobieranie książek, usuwanie nagłówków, normalizacja, podział na zbiory, losowanie fragmentów
- gasub/cipher.py – szyfr podstawieniowy, kodowanie liter na indeksy, metryki poprawności
- gasub/ngrams.py – trening i zapis modelu n-gramowego (.npz)
- gasub/ga.py – konfiguracja i pętla GA w DEAP
- gasub/experiments.py – zadania, metryki, zapis JSON, równoległe wykonanie prób (multiprocessing)
- encrypt.py, crack.py – narzędzia CLI (szyfrowanie i atak)
- run_experiments.py – próba ataku wg kryterium projektu i eksperymenty dodatkowe

4.2. Wydajność funkcji przystosowania

Szyfrogram jest kodowany raz jako tablica indeksów, odszyfrowanie to indeksowanie tablicą podstawień, a indeksy n-gramów powstają z przesuniętych widoków tablicy. Suma logarytmów to jedno indeksowanie tablicy modelu i redukcja. Jedna ocena 500-znakowego szyfrogramu trwa ok. 4 µs, a cały przebieg GA ($\approx 60\,000$ ocen) – około sekundy. Sto prób oceny końcowej wykonuje się równolegle na 12 rdzeniach w kilkanaście sekund.

4.3. Fragmenty kodu

Listing 4.1 pokazuje szyfrowanie na tablicach indeksów i metrykę sukcesu, listing 4.2 – wektorową funkcję przystosowania, a listing 4.3 – rejestrację operatorów w Toolbox i rdzeń pętli ewolucyjnej.

```python
def substitution_table(key) -> np.ndarray:
    """Key extended with the space symbol (26 -> 26) as an index table."""
    table = np.empty(N_LETTERS + 1, dtype=np.int64)
    table[:N_LETTERS] = key
    table[SPACE] = SPACE
    return table

def apply_key(idx: np.ndarray, key) -> np.ndarray:
    return substitution_table(key)[idx]

def encrypt(text: str, enc_key) -> str:
    return decode(apply_key(encode(text), enc_key))

def decrypt(ciphertext: str, dec_key) -> str:
    return decode(apply_key(encode(ciphertext), dec_key))

def letter_accuracy(plain_idx, decrypted_idx) -> float:
    """Fraction of letter positions (spaces excluded) matching the
    plaintext."""
    mask = plain_idx != SPACE
    return float(np.mean(plain_idx[mask] == decrypted_idx[mask]))
```

Listing 4.1: Szyfr podstawieniowy na tablicach indeksów i poprawność literowa (fragment gasub/cipher.py)

```python
class FitnessFunction:
    def __init__(self, model: NgramModel, ciphertext_idx: np.ndarray):
        self.model = model
        self.cipher_idx = np.asarray(ciphertext_idx, dtype=np.int64)
        self.n, self.V = model.n, model.V
        L = len(self.cipher_idx) - self.n + 1
        self.windows = [slice(k, k + L) for k in range(self.n)]
        self.evaluations = 0

    def __call__(self, key) -> float:
        self.evaluations += 1
        dec = substitution_table(key)[self.cipher_idx]   # decrypt
        idx = dec[self.windows[0]].copy()                # n-gram indices
        for window in self.windows[1:]:
            idx *= self.V
            idx += dec[window]
        return float(self.model.logp[idx].sum())         # sum of log10 P
```

Listing 4.2: Funkcja przystosowania $F(d) = \sum_i \log_{10} P(g_i)$ dla ustalonego szyfrogramu (fragment gasub/ngrams.py)

```python
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
tb = base.Toolbox()
tb.register("individual", lambda: creator.Individual(rng.sample(range(26),
    26)))
tb.register("evaluate", lambda ind: (fitness(ind),))     # F(d): n-gram
    log-likelihood
tb.register("mate", tools.cxPartialyMatched)             # PMX
tb.register("mutate", tools.mutShuffleIndexes, indpb=cfg.indpb)
tb.register("select", tools.selTournament, tournsize=cfg.tournament)
```

```
9
10 for generation in range(1, cfg.generations + 1):
11     elite = [tb.clone(e) for e in tools.selBest(pop, cfg.elite)]
12     offspring = tb.select(pop, len(pop) - cfg.elite)
13     offspring = algorithms.varAnd(offspring, tb, cfg.cxpb, cfg.mutpb)
14     for ind in offspring:
15         if not ind.fitness.valid:
16             ind.fitness.values = tb.evaluate(ind)
17     pop = elite + offspring
18     hof.update(pop)
```

Listing 4.3: Konfiguracja DEAP i pętla ewolucyjna (fragment gasub/ga.py)

4.4. Odtwarzalność

Każda próba jest wyznaczona przez ziarno (fragment, klucz, przebieg GA), a wyniki są zapisywane w plikach JSON z pełną konfiguracją.

Rozdział 5

Wyniki

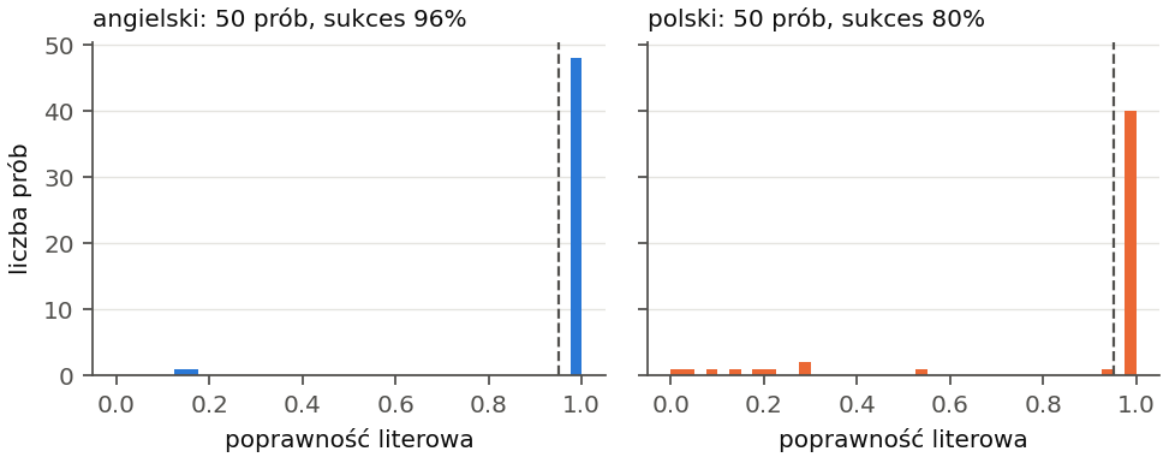
W tabelach: N – liczba prób, poprawność – poprawność literowa Acc (średnia $\pm$ odchylenie standardowe, mediana, minimum), sukces – odsetek prób z Acc $> 0{,}95$, $F \geq F^*$ – odsetek prób, w których znaleziony klucz ma przystosowanie nie gorsze niż klucz prawdziwy, czas – średni czas jednej próby na jednym rdzeniu CPU. Szyfrogramy mają 500 znaków i składają się z samych liter, o ile nie zaznaczono inaczej.

5.1. Próba ataku według kryterium projektu

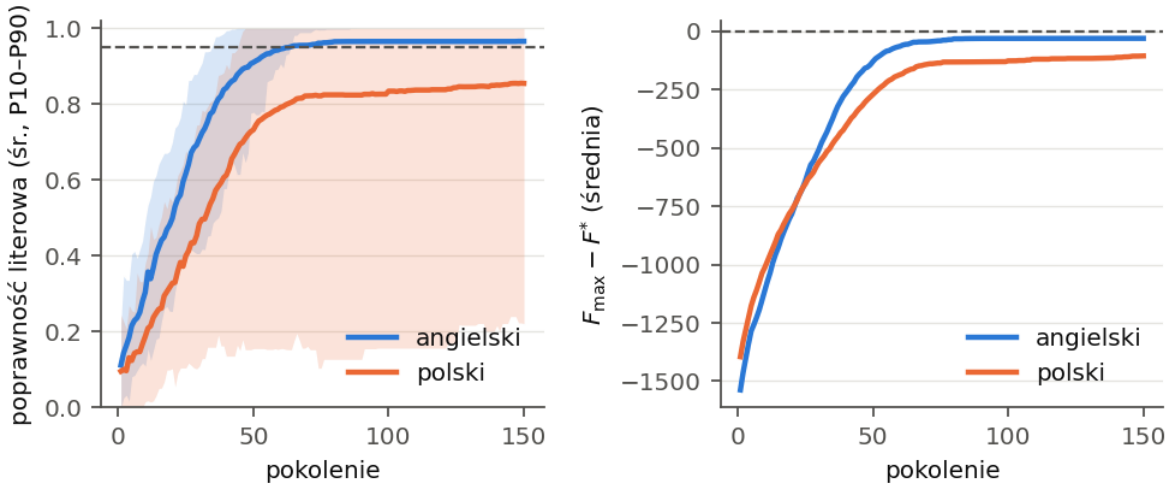
GA zaatakował 50 szyfrogramów angielskich i 50 polskich (tabela 5.1, rysunki 5.1 i 5.2). Kryterium Acc $> 0{,}95$ spełniono w 96 % prób dla języka angielskiego i 80 % dla polskiego. Średnia poprawność wyniosła odpowiednio 0,966 i 0,855, mediana 1,000 i 1,000, a najgorsza próba 0,144 i 0,022. Wszystkie 12 porażek ma $F < F^*$, czyli populacja zbiegła przedwcześnie do lokalnego maksimum. Próg 95 % był osiągany po medianie 47 pokoleń, a średni czas próby to 1,02 s.

Tabela 5.1: Próba ataku wg kryterium projektu: GA, szyfrogramy 500-znakowe (same litery).

wariant	język	N	poprawność (śr. $\pm$ odch.)	mediana	min	sukces	$F \geq F^*$	czas [s]
GA	EN	50	0,966 $\pm$ 0,169	1,000	0,144	96 %	96 %	1,02
GA	PL	50	0,855 $\pm$ 0,316	1,000	0,022	80 %	80 %	1,01



Rysunek 5.1: Rozkład poprawności literowej w próbach głównego eksperymentu (linia przerywana: próg 0,95).

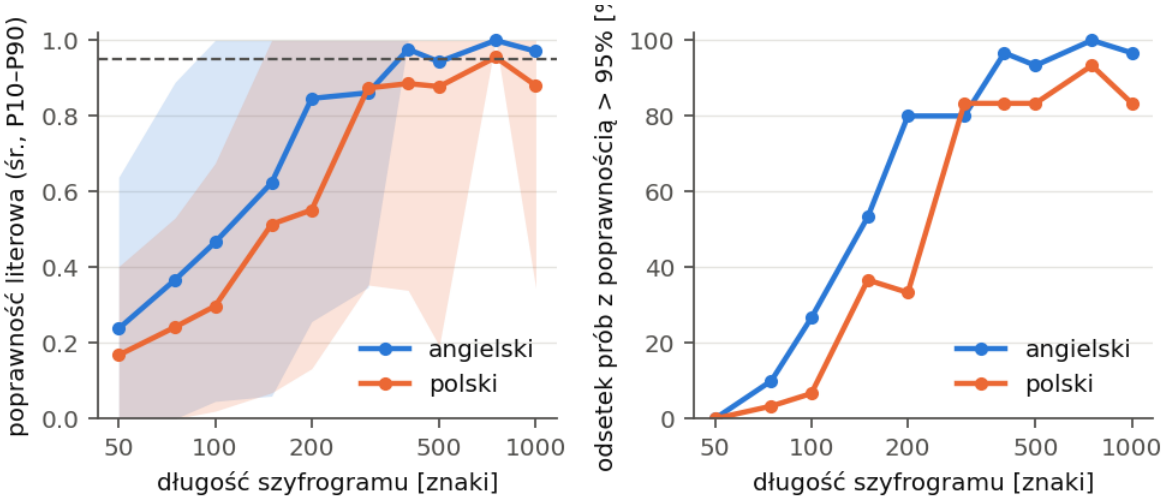


Rysunek 5.2: Zbieżność GA: poprawność literowa najlepszego osobnika w kolejnych pokoleniach (średnia i pasmo P10–P90) oraz różnica między najlepszym przystosowaniem a przystosowaniem klucza prawdziwego.

Kryterium projektu jest więc spełnione dla języka angielskiego z zapasem, a dla polskiego – w 80 % prób. Teksty polskie po usunięciu diakrytyków są trudniejsze (bogatsza fleksja, mniejszy korpus treningowy).

5.2. Wpływ długości szyfrogramu

Odsetek sukcesów co najmniej 90 % po raz pierwszy osiągnięto przy $L = 400$ znakach dla języka angielskiego i $L = 750$ dla polskiego, a połowa prób kończyła się sukcesem od $L = 150$ (EN) i $L = 300$ (PL) (rysunek 5.3, tabela 5.2). Krótki szyfrogram zawiera za mało n-gramów, by model językowy odróżnił klucz prawdziwy od kluczy dających tekst pozornie poprawny.



Rysunek 5.3: Wpływ długości szyfrogramu na poprawność literową (średnia, pasmo P10–P90) i odsetek sukcesów.

Tabela 5.2: Wpływ długości szyfrogramu L (GA, 30 prób na język).

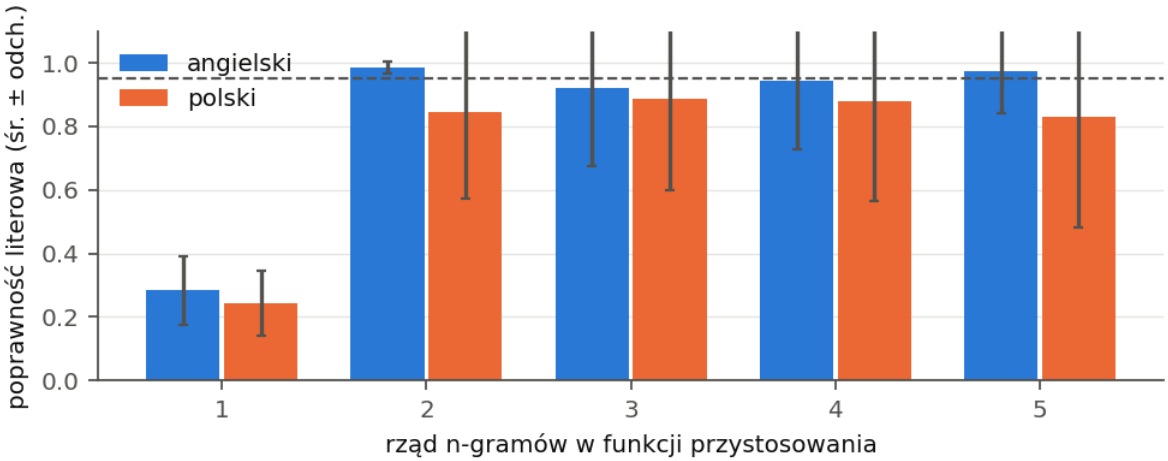
wariant	język	N	poprawność (śr. $\pm$ odch.)	mediana	min	sukces	$F \geq F^*$	czas [s]
$L = 50$	EN	30	0,238 $\pm$ 0,278	0,160	0,000	0 %	7 %	0,93
$L = 75$	EN	30	0,367 $\pm$ 0,314	0,287	0,000	10 %	13 %	0,96
$L = 100$	EN	30	0,466 $\pm$ 0,391	0,310	0,000	27 %	27 %	0,97
$L = 150$	EN	30	0,623 $\pm$ 0,422	0,987	0,000	53 %	53 %	1,02
$L = 200$	EN	30	0,846 $\pm$ 0,318	1,000	0,090	80 %	80 %	1,05
$L = 300$	EN	30	0,861 $\pm$ 0,297	1,000	0,050	80 %	80 %	1,20
$L = 400$	EN	30	0,976 $\pm$ 0,131	1,000	0,282	97 %	97 %	1,09
$L = 500$	EN	30	0,943 $\pm$ 0,216	1,000	0,144	93 %	93 %	1,19
$L = 750$	EN	30	1,000 $\pm$ 0,001	1,000	0,995	100 %	100 %	1,20
$L = 1000$	EN	30	0,971 $\pm$ 0,158	1,000	0,135	97 %	97 %	1,27
$L = 50$	PL	30	0,168 $\pm$ 0,152	0,140	0,000	0 %	13 %	0,93
$L = 75$	PL	30	0,242 $\pm$ 0,257	0,200	0,000	3 %	13 %	0,96
$L = 100$	PL	30	0,298 $\pm$ 0,278	0,250	0,010	7 %	7 %	0,99
$L = 150$	PL	30	0,514 $\pm$ 0,412	0,327	0,000	37 %	33 %	1,04
$L = 200$	PL	30	0,552 $\pm$ 0,362	0,427	0,025	33 %	33 %	1,05
$L = 300$	PL	30	0,874 $\pm$ 0,291	1,000	0,100	83 %	83 %	1,09
$L = 400$	PL	30	0,886 $\pm$ 0,280	1,000	0,065	83 %	83 %	1,10
$L = 500$	PL	30	0,878 $\pm$ 0,311	1,000	0,022	83 %	83 %	1,16
$L = 750$	PL	30	0,955 $\pm$ 0,171	1,000	0,244	93 %	93 %	1,20
$L = 1000$	PL	30	0,880 $\pm$ 0,276	1,000	0,051	83 %	83 %	1,25

5.3. Rząd n-gramów w funkcji przystosowania

Unigramy nie wystarczają: wskaźnik $F \geq F^*$ wynosi dla nich 100 %, więc GA znajduje maksimum funkcji, a porażka leży po stronie modelu (tabela 5.3, rysunek 5.4). Dla $n \geq 2$ różnice między rzędami mieszczą się w granicach błędu losowego, a o porażkach decyduje przeszukiwanie. Wraz ze wzrostem n model lepiej wyróżnia tekst prawdziwy, ale tablica rośnie i staje się rzadka.

Tabela 5.3: Wpływ rzędu n-gramów w funkcji przystosowania (500 znaków, GA).

wariant	język	N	poprawność (śr. ± odch.)	mediana	min	sukces	$F \geq F^*$	czas [s]
$n = 1$	EN	30	0,284 ± 0,109	0,284	0,076	0 %	100 %	0,97
$n = 1$	PL	30	0,242 ± 0,102	0,226	0,070	0 %	100 %	0,97
$n = 2$	EN	30	0,987 ± 0,019	0,998	0,942	93 %	90 %	1,03
$n = 2$	PL	30	0,845 ± 0,271	1,000	0,098	63 %	67 %	1,03
$n = 3$	EN	30	0,920 ± 0,245	1,000	0,094	90 %	90 %	1,10
$n = 3$	PL	30	0,885 ± 0,286	1,000	0,038	83 %	83 %	1,09
$n = 4$	EN	30	0,943 ± 0,216	1,000	0,144	93 %	93 %	1,16
$n = 4$	PL	30	0,878 ± 0,311	1,000	0,022	83 %	83 %	1,17
$n = 5$	EN	30	0,975 ± 0,135	1,000	0,258	97 %	97 %	1,28
$n = 5$	PL	30	0,831 ± 0,348	1,000	0,068	80 %	80 %	1,27



Rysunek 5.4: Poprawność literowa w zależności od rzędu n-gramów w funkcji przystosowania.

5.4. Szyfrogram z zachowanymi odstępami

Dla języka angielskiego poprawność wyniosła 0,943 bez odstępów i 1,000 z odstępami (sukces 93 % wobec 100 %), a dla polskiego odpowiednio 0,878 i 1,000 (83 % wobec 100 %), jak pokazuje tabela 5.4. Jawne odstępy dostarczają modelowi granic słów, chociaż 500-znakowy szyfrogram z odstępami zawiera o ok. 18 % mniej liter.

Tabela 5.4: Szyfrogram z jawnymi odstępami vs same litery (500 znaków, GA).

wariant	język	N	poprawność (śr. ± odch.)	mediana	min	sukces	$F \geq F^*$	czas [s]
bez odstępów	EN	30	0,943 ± 0,216	1,000	0,144	93 %	93 %	1,15
bez odstępów	PL	30	0,878 ± 0,311	1,000	0,022	83 %	83 %	1,20
z odstępami	EN	30	1,000 ± 0,000	1,000	0,998	100 %	100 %	1,18
z odstępami	PL	30	1,000 ± 0,002	1,000	0,990	100 %	100 %	1,19

5.5. Przykład działania: szyfrowanie i atak

Listing 5.1 pokazuje zaszyfrowanie krótkiej wiadomości losowym kluczem, a listing 5.2 – atak na otrzymany szyfrogram. Odszyfrowany tekst jest w pełni zgodny z wiadomością.

```
$ python3 encrypt.py --spaces -o examples/ciphertext_en.txt
    examples/message_en.txt
Encryption key: MYNBIQPZJTLRDVCEUWKHFGSXAO
Plaintext (217 characters):
THE FLAG IS HIDDEN IN THE OLD LIGHTHOUSE ON THE NORTHERN SHORE MEET ME THERE
    AT MIDNIGHT AND BRING THE SECOND KEY THE GUARDS CHANGE SHIFTS AT TWELVE
    SO WE WILL HAVE A FEW MINUTES TO OPEN THE DOOR BEFORE ANYONE NOTICES

Ciphertext (saved to examples/ciphertext_en.txt):
HZI QRMP JK ZJBBIV JV HZI CRB RJPZHZCFKI CV HZI VCWHZIWV KZCWI DIIH DI HZIWI
    MH DJBVJPZH MVB YWJVP HZI KINCVB LIA HZI PFMWBK NZMVPI KZJQHK MH HSIRGI
    KC SI SJRR ZMGI M QIS DJVFHIK HC CEIV HZI BCCW YIQCWI MVACVI VCHJNIK
```

Listing 5.1: Szyfrowanie wiadomości losowym kluczem (encrypt.py)

```
$ python3 crack.py --lang en --spaces examples/ciphertext_en.txt
Ciphertext (217 characters):
HZI QRMP JK ZJBBIV JV HZI CRB RJPZHZCFKI CV HZI VCWHZIWV KZCWI ... (pełny
    szyfrogram jak w listingu wyżej)

GA: 150 generations, 55233 evaluations, 0.8 s, F = -795.2
Decryption key (cipher -> plain): YDOMPUVTEISKACQGFLWJZNRXBH
Corresponding encryption key:     MYNBIQPZJTLRDVCEOWKHFGSXAU

Decrypted text:
THE FLAG IS HIDDEN IN THE OLD LIGHTHOUSE ON THE NORTHERN SHORE MEET ME THERE
    AT MIDNIGHT AND BRING THE SECOND KEY THE GUARDS CHANGE SHIFTS AT TWELVE
    SO WE WILL HAVE A FEW MINUTES TO OPEN THE DOOR BEFORE ANYONE NOTICES
```

Listing 5.2: Atak na szyfrogram z listingu 5.1 (crack.py)

Odtworzony klucz szyfrujący różni się od prawdziwego tylko obrazami liter Q i Z, które w wiadomości nie występują: z punktu widzenia szyfrogramu ich podstawienia są nieodróżnialne, więc atak nie może (i nie musi) ich rozstrzygnąć, a poprawność literowa odczytu wynosi 100 %.

Rozdział 6

Wnioski

- Zrealizowano wszystkie etapy projektu oraz zaplanowane eksperymenty.
- Kryterium projektu spełniono w 96 % prób dla języka angielskiego i 80 % dla polskiego, w czasie około sekundy na szyfrogram.
- Wszystkie porażki to przedwczesna zbieżność populacji, a nie błąd modelu językowego.
- Skuteczność zależy przede wszystkim od długości szyfrogramu (co najmniej 90 % sukcesów od ok. 400 znaków dla języka angielskiego i 750 dla polskiego).
- Funkcja unigramowa nie wystarcza, a rzędy $n \geq 2$ dają przy 500 znakach zbliżone wyniki.
- Jawne odstępy między słowami ułatwiają atak (100 % sukcesów w obu językach).

6.1. Ograniczenia

- **Dobór korpusu.** Model i teksty testowe pochodzą z literatury XIX/XX wieku. Dla tekstów współczesnych, technicznych lub pełnych nazw własnych rozkład n-gramów różni się, co może obniżyć skuteczność.
- **Normalizacja języka polskiego.** Usunięcie znaków diakrytycznych zlewa pary liter (z/ż/ź, a/ą), więc „polski” w naszym modelu jest 26-literową aproksymacją.
- **Zakres modelu zagrożeń.** Wyniki dotyczą monoalfabetycznego podstawienia w znanym języku.

Bibliografia

[1] S. Picek et al., „Evolutionary algorithms for cryptanalysis: A survey”, *Applied Soft Computing*, 2016.

[2] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, C. Gagné, „DEAP: Evolutionary Algorithms Made Easy”, *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012; repozytorium https://github.com/DEAP/deap.

[3] Project Gutenberg, *Free eBooks*, https://www.gutenberg.org, 2024.

[4] Fundacja Wolne Lektury, *Wolne Lektury*, https://wolnelektury.pl, 2024.

Dodatek A

Macierz wkładu

Tabela A.1: Macierz wkładu członków zespołu.

członek zespołu	zakres prac
Radosław Rolka	szkielet projektu, korpusy tekstów, szyfr podstawieniowy, protokół eksperymentów, wyniki ataku, sprawozdanie PDF
Weronika Wojtas	model n-gramowy, algorytm genetyczny, narzędzia CLI, eksperymenty dodatkowe

Spis rysunków

5.1 Rozkład poprawności literowej w próbach głównego eksperymentu (linia przerywana: próg 0,95). 15
5.2 Zbieżność GA: poprawność literowa najlepszego osobnika w kolejnych pokoleniach (średnia i pasmo P10–P90) oraz różnica między najlepszym przystosowaniem a przystosowaniem klucza prawdziwego. 15
5.3 Wpływ długości szyfrogramu na poprawność literową (średnia, pasmo P10–P90) i odsetek sukcesów. 16
5.4 Poprawność literowa w zależności od rzędu n-gramów w funkcji przystosowania. 17

Spis tabel

3.1 Korpusy tekstowe użyte w eksperymentach (liczba liter po normalizacji do 26 liter). 8
3.2 Konfiguracja algorytmu genetycznego. 10

5.1 Próba ataku wg kryterium projektu: GA, szyfrogramy 500-znakowe (same litery). 14
5.2 Wpływ długości szyfrogramu L (GA, 30 prób na język). 16
5.3 Wpływ rzędu n-gramów w funkcji przystosowania (500 znaków, GA). 17
5.4 Szyfrogram z jawnymi odstępami vs same litery (500 znaków, GA). 17

A.1 Macierz wkładu członków zespołu. 21